CMPE-273 · SPRING 2026 · FINAL PROJECT

Mesh Control

Self-Healing Microservice Mesh

An autonomous SRE for the microservice graph.

VK **Vineet Kumar**
SJSU ID: 019140433

SS **Samved Sandeep Joshi**
SJSU ID: 019100107

GC **Girith Choudhary**
SJSU ID: 018281744

Instructor: **Prof. Rakesh Ranjan** · San José State University · Department of Computer Engineering

THE PROBLEM

Every microservice outage still waits on a tired human.

Production breaks. PagerDuty fires. An SRE wakes at 3 AM, opens five dashboards, runs `kubectl get pods`, traces a request, restarts the bad service.

Industry median MTTR: **27 minutes** (Google SRE 2023 incident report)

01

The five steps are mechanical.

Read telemetry, walk the dependency graph, pick a remediation, execute, verify. None require irreplaceable judgment — all depend on human reaction time.

02

Dashboards show what's red — not why.

Datadog, New Relic, Grafana surface failures. The SRE still has to look at five red services and decide which is the cause and which four are victims.

03

The bottleneck isn't intelligence — it's bandwidth.

One human responder, five dashboards, 27 minutes. The cost: lost revenue, eroded trust, burned-out engineers.

What if the SRE never went to sleep?

PERSONA

Who this is for

— and what part of their job it replaces.

ON-CALL SRE

Lina

Senior SRE · payments company

- Pages at 03:11 for elevated 5xx on /checkout.
- Spends 18 minutes proving payments — not order — is the cause.
- Restarts payments. Sleeps at 04:23. Standup at 09:00.

Mesh Control does this in **5 seconds** while she sleeps.

ENG DIRECTOR

Marcus

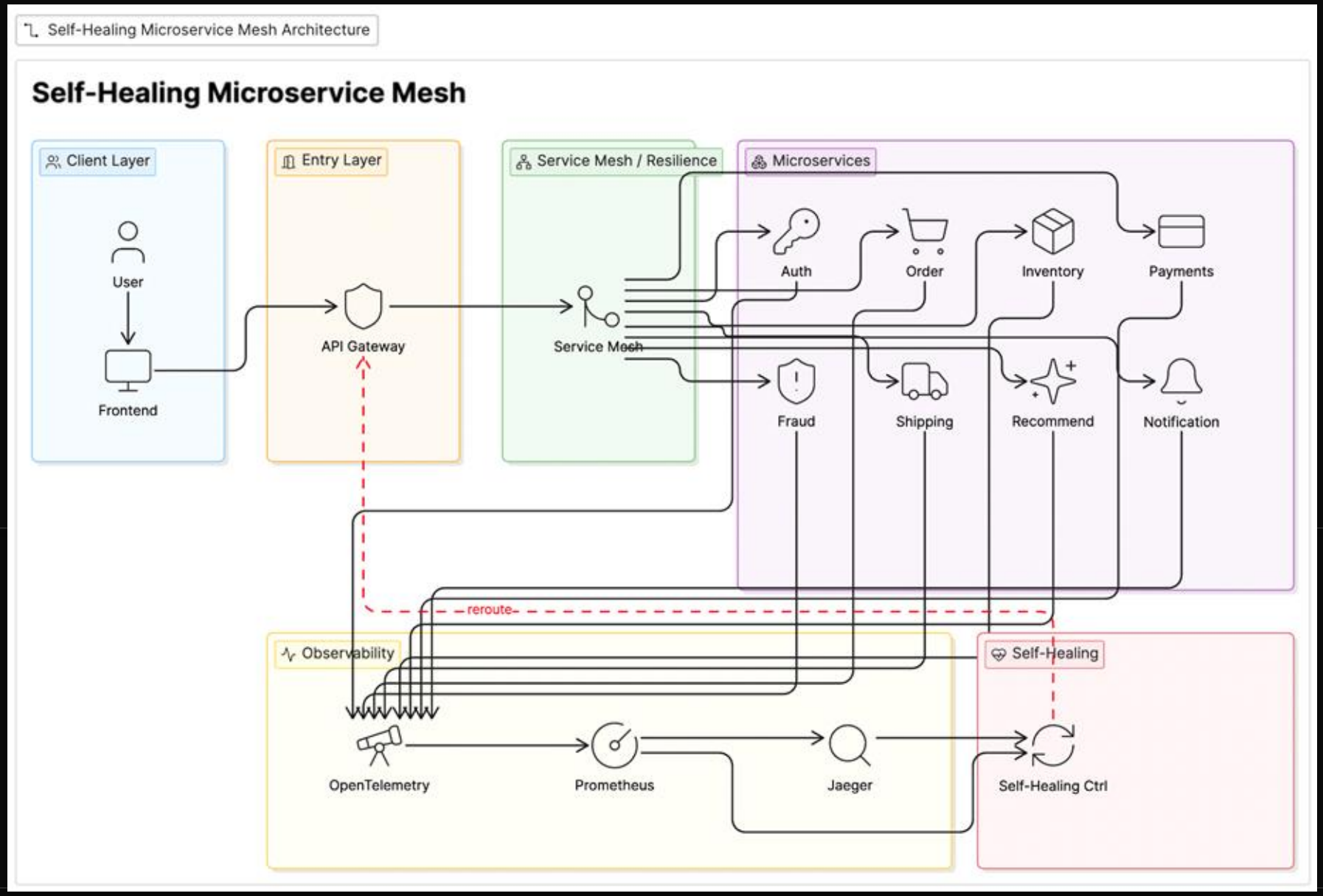
VP Engineering · Series B startup

- Three SREs covering 60 services. Two are burning out.
- Every postmortem: "root-cause ID took too long."
- Needs to either hire 5 more humans or bound the toil.

A 4-verb action surface a CTO can read in **30 seconds** — auditable.

We don't replace SREs. We replace the part of their job they hate.

Architecture



WHY THIS WORKS

Three properties that work in production

— not just in demos.

01

Symptom vs. root cause separation

Most dashboards show what's red. Ours walks the dependency graph and identifies the deepest failing service — the one with no failing dependency below it. The same call an experienced SRE makes, done in two seconds.

02

Bounded autonomous action

The agent can only execute four verbs. Even if the LLM hallucinates "delete production database," the allowlist rejects it silently and the rule engine takes over.

03

Same code in dev and prod

Chaos primitives are baked into every service. A staging drill and a production fault use literally the same observation pipeline. You're not testing a parallel test harness — you're testing the production system.

Most "AI in production" stories skip the safety question. We led with it.

USE CASES

Where it earns its keep

across the software lifecycle.

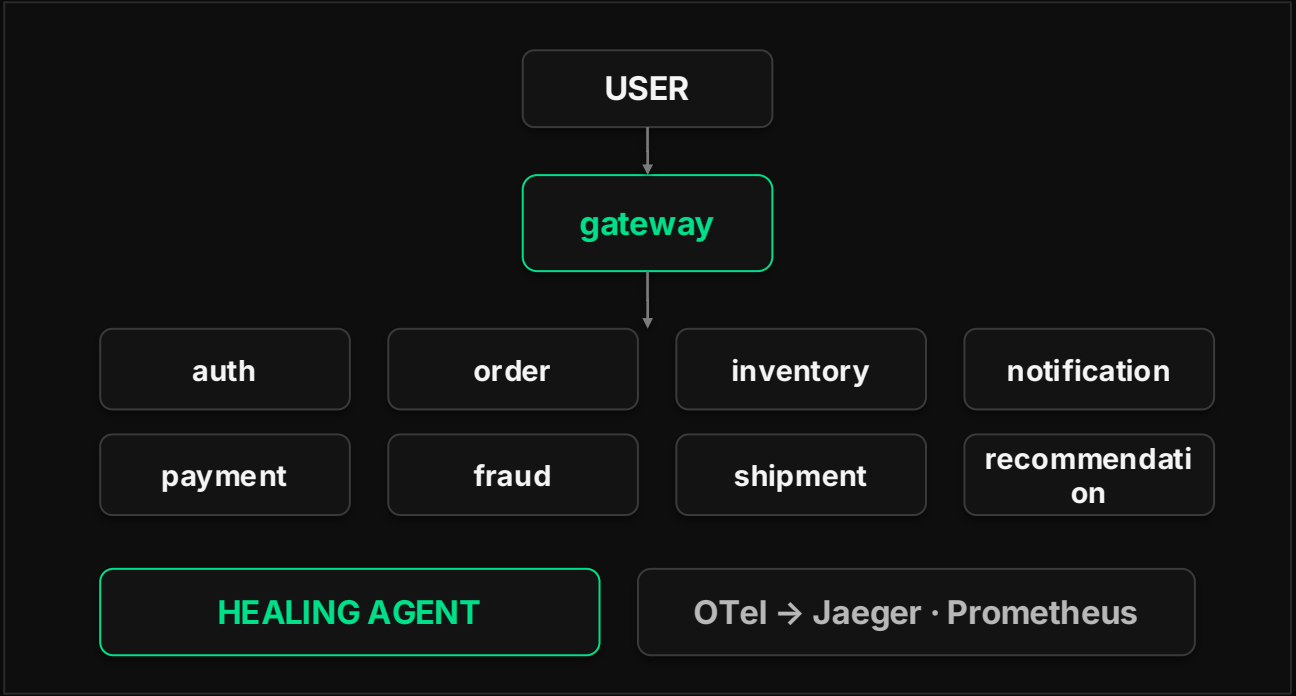
STAGE	FIT	WHAT IT DOES	WIN
Resilience testing	STRONG	Inject error/latency/grey failures into any service. Watch blast radius spread, agent diagnose, recovery within 10 s.	*Replaces a 30-min manual chaos drill with one click.*
Pre-prod gameday	STRONG	New SREs train on real failure scenarios. Architects validate fallback paths before promotion.	*"What happens when payments goes down?" — answerable in 30 s, not a 2-hour table-top.*
CI/CD release gate	MEDIUM	Deploy → run scripted flows → inject one failure → verify auto-heal within SLA → promote.	*Catches "the deploy broke our fallback" before it reaches users.*
Production tier-1	STRONG	Agent runs as infrastructure. Handles the well-defined 80% — restarts, circuit breakers, fallback activation. Pages a human only when allowlist is exhausted.	*Drops MTTR from 27 min to 5 s for everything in scope.*
Stakeholder comms	STRONG	Plain-English incident summaries. Blast-radius diagrams. Token-by-token LLM reasoning visible.	*An exec asks "what happened?" — screenshot the incident card.*

Chaos testing is the obvious use. Tier-1 production response is the bigger one.

ARCHITECTURE

Three planes of communication.

One agent. Sixteen containers.



The three planes

01 Request plane `gRPC`

synchronous, caller → callee. Real work flows here.

02 Side channel `NATS pub/sub`

telemetry + failure injection. Agent is a subscriber, not a proxy.

03 Out-of-band trace `OTLP/gRPC`

distributed tracing. No business logic depends on it.

If the healing agent crashes, real user requests keep flowing — they just stop being auto-remediated. *Same architectural pattern as Kubernetes controllers.*

DEMO

injecting chaos in 3 … 2 … 1